



ЛЕКЦИЯ 19

Нарушение целостности обучающих данных и артефактов

poisoning · backdoor · supply-chain

Машинное обучение и безопасность систем

Магистратура НИУ ВШЭ (МИЭМ) · ОП «Информационная безопасность и технологии ИИ»

Автор: Силаев Ю. В. · Время изучения: ~67 минут · 52 слайда



О чём эта лекция за минуту

2 / 52

Часть 1 · Введение

Угроза смещается со входа модели на её обучение и цепочку поставки артефактов

До сих пор мы защищали модель в момент её *применения*: в лекции L18 противник подавал специально искажённый вход уже обученной модели. Эта лекция – про другую поверхность атаки. Здесь под удар попадает то, на чём и из чего модель построена: обучающая выборка, процесс её разметки и сама цепочка поставки ML-артефактов – внешние датасеты, претренированные веса, репозитории моделей, рёр-зависимости.

КЛЮЧЕВОЙ ТЕЗИС

Если нарушена целостность обучающих данных или цепочки поставки артефактов, модель может выучить вредоносные закономерности даже при формально корректном обучении и высокой метрике на тесте.



Учебные результаты

- различать случайный шум разметки и целенаправленное отравление обучающих данных;
- объяснять, что такое data poisoning и backdoor на концептуальном уровне;
- понимать идею backdoor как скрытого триггера нежелательного поведения модели;
- описывать supply-chain риски: внешние датасеты, претренированные веса, репозитории, pip-пакеты;
- объяснять, почему высокая итоговая метрика не исключает poisoning- или backdoor-риска;
- формулировать минимальные требования к проверке обучающих данных и используемых артефактов в ИБ-контексте.



Что нужно знать до начала

4 / 52

Часть 1 · Введение

Лекция опирается на три темы

L18 · Модель угроз ML

Состязательные воздействия и evasion. Термин «threat modeling для ML» вводится там и здесь используется без переопределения.

L05 · Утечка данных (leakage)

Концептуальная типология leakage и схемы валидации. Poisoning мы будем сравнивать с leakage как граничным случаем.

L14 · Transfer learning

Перенос претренированных backbone-моделей. Именно эти артефакты окажутся точкой supply-chain риска.



Сквозная линия «threat modeling для ML» проходит через четыре лекции блока 3



Эта лекция – третья компонента. L17 разбирал изменения, которые среда вносит сама; L18 – противника на входе уже обученной модели. L19 показывает третий вектор: противник действует на этапе обучения и через артефакты. L20 закрывает картину приватностью и сводит всё в итоговую оценку надёжности.



План лекции

6 / 52

Часть 1 · Введение

7 частей · 52 слайда · около 67 минут самостоятельного изучения

Часть	Содержание	Слайды	Время
1	Введение	1-6	~8 мин
2	Мотивация: почему данные и артефакты – поверхность атаки	7-11	~6 мин
3	Основное содержание: poisoning · backdoor · supply-chain	12-40	~38 мин
4	Связки с практикой ИБ (CIC-IDS-2018)	41-47	~9 мин
5	Итоги	48-50	~4 мин
6	Источники для углубления	51	~1 мин
7	Глоссарий	52	~1.5 мин



Датасет – не нейтральный объект

7 / 52

Часть 2 · Мотивация

В безопасности данные приходят из множества неоднородных и частично внешних источников

В учебных задачах датасет часто воспринимается как данность: вот таблица объектов, вот метки, обучаем модель. В реальной ИБ-системе всё иначе. Данные собираются **из множества источников** – собственная телеметрия, внешние фиды угроз, публичные корпуса, crowdsourcing-разметка, автоматически собранные логи. Часть этих источников вы не контролируете, а часть вообще принадлежит третьим сторонам.

ГЛАВНАЯ ИДЕЯ ЧАСТИ

Если данные и артефакты приходят из источников, которым вы не доверяете полностью, то **процесс формирования датасета и цепочка поставки модели становятся частью поверхности атаки** – наравне со входом модели в момент применения. Противник, способный повлиять на обучающие данные или на используемые артефакты, влияет на всю последующую эксплуатацию системы.



Ловушка доверия по умолчанию

8 / 52

Часть 2 · Мотивация

Две установки, с которыми инженер чаще всего приходит к задаче – и обе опасны

⚠ ЧАСТАЯ ОШИБКА

«**Датасет нейтрален**». Раз данные собраны и размечены, им можно доверять как объективному факту.

«**Претренированные веса доверенные**». Раз модель скачана из популярного репозитория и показывает хорошее качество – значит, с ней всё в порядке.

ПОЧЕМУ ЭТО НЕВЕРНО

Обе установки делают целостность данных и артефактов **слепой зоной**. Никакая часть стандартного пайплайна – обучение, валидация, замер метрики – не проверяет, откуда данные взялись и не подменён ли backbone. Если источник скомпрометирован, высокое качество на тесте лишь маскирует проблему: модель уверенно делает ровно то, чему её научили отравленные данные.



Типичные каналы поступления данных и артефактов в ИБ-системе обнаружения

ИБ-КЕЙС SOC обучает детектор сетевых атак – четыре канала данных и артефактов, где теряется доверие:

Crowdsourcing-разметка

Метки ставят внешние аннотаторы разного уровня; часть меток систематически смещена или подделана.

Автосбор логов

События пишутся автоматически без верификации; в корпус попадает нехарактерное для реальной эксплуатации.

Публичные фиды и корпуса

Внешние датасеты угроз с неизвестной историей подготовки и происхождением строк.

Репозитории весов

Претренированный backbone или embedding скачан из публичного репозитория без проверки происхождения.



Чем атака на обучение принципиально опаснее разовой ошибки на входе

Атака на входе (L18)

- Действует на один запрос
- Модель остаётся исправной
- Эффект локален и обратим: убрали вход – эффект исчез

Атака на обучение (L19)

- Запечена в веса модели навсегда
- Срабатывает на всех будущих запусках
- Эффект глобален: убрать можно только переобучением на чистых данных

Вывод: одна отравленная строка или один подменённый артефакт влияет не на отдельный ответ, а на **всю последующую эксплуатацию модели**. Поэтому целостность обучения требует отдельного контроля, а не только хороших метрик.



Что нас ждёт впереди

11 / 52

Часть 2 · Мотивация

Основное содержание развёрнуто в трёх блоках – по нарастанию незаметности угрозы

1

Data poisoning

Загрязнение обучающих данных: искажение меток (label) и признаков (feature), целенаправленное смещение состава корпуса.

заметнее

2

Backdoor

Скрытый триггер: модель ведёт себя нормально, пока на входе не появится особый шаблон, запускающий нежелательное поведение.

скрытнее

3

Supply-chain

Подмена самих артефактов: претренированные веса, репозитории моделей, pip-зависимости, сторонние backbone из transfer learning.

*почти
невидимо*

Граница с L18 – первый же слайд основного содержания: evasion (вход) против poisoning (обучение).



Перед содержанием – чёткое различие двух угроз из L18 и L19, которые легко перепутать

EVASION · L18

Фаза: применение обученной модели

Что нарушает: вход в момент запроса

Модель: остаётся неизменной

Аналогия: подделка пропуска на проходной

POISONING · L19

Фаза: обучение модели (и сборка артефактов)

Что нарушает: обучающие данные и веса

Модель: выучивает вредоносное поведение

Аналогия: подкуп инструктора на этапе обучения охраны

Обе угрозы – часть единой модели угроз ML (threat modeling, термин из L18). Различие – в фазе: evasion атакует уже готовую модель на входе, poisoning подрывает её ещё на обучении.



1

БЛОК 1 ИЗ 3

Data poisoning

Загрязнение обучающих данных: что происходит, когда в обучающую выборку целенаправленно вносят искажения – в метки, в признаки или в состав корпуса.



Концептуальное определение – без рецептов построения атак

ОПРЕДЕЛЕНИЕ

Data poisoning (отравление данных)

целенаправленное внесение в обучающую выборку примеров, меток или искажений, рассчитанных на то, чтобы модель выучила нужную противнику закономерность или потеряла способность распознавать определённые объекты.

Ключевое слово – **целенаправленное**. Речь не о случайных дефектах сбора, а о воздействии, у которого есть умысел и цель. Отравление может затрагивать **метки** (label poisoning), **признаки** (feature poisoning) или **состав корпуса** – пропорции классов и источников. Все три механизма разберём дальше в этом блоке.



Самое важное различие блока: не всякая грязь в данных – атака

Случайный шум разметки

- Возникает сам – усталость аннотатора, неоднозначные случаи
- Распределён более-менее равномерно, без умысла
- Снижает качество, но предсказуемо и симметрично

Целенаправленное poisoning

- Вносится умышленно, с конкретной целью
- Сконцентрировано там, где даёт нужный противнику эффект
- Может почти не трогать общую метрику, меняя поведение точно

Граничный случай – leakage (как в L05). Утечка тоже искажает обучение, но непреднамеренно: данные «подсматривают» будущее. Poisoning отличается умыслом. Признак тот же – модель учится не тому; механизм разный.



Первый механизм отравления – атака на разметку, а не на сами данные объектов

ОПРЕДЕЛЕНИЕ

Label poisoning

часть обучающих объектов получает неверные метки так, чтобы сдвинуть границу принятия решения в сторону, выгодную противнику.

Сами признаки объектов остаются настоящими – искажена только разметка. **В ИБ-сценарии:** часть вредоносных сетевых соединений в обучающем корпусе помечается как Benign. Модель «привыкает», что трафик с такими характеристиками безопасен, и в эксплуатации пропускает похожие атаки. На общей метрике это может быть почти незаметно, если отравлена лишь узкая область пространства признаков.



Второй механизм – метки верны, но искажены значения признаков объектов

ОПРЕДЕЛЕНИЕ

Feature poisoning

значения признаков части обучающих объектов искажаются так, чтобы модель связала вредоносное поведение с «безопасным» паттерном признаков – или наоборот.

Здесь метка может быть правильной, но **сам объект препарирован**. В ИБ-сценарии: в обучающие записи об атаках систематически добавляется некоторый признак-«маркер» (например, специфическое значение поля), которого в реальных атаках нет. Модель выучивает ложную зависимость: «атака = наличие маркера». В эксплуатации атаки без маркера проходят мимо детектора. Это подводит нас к идее **скрытого триггера** – её разберём в блоке 2 (backdoor).



Третий механизм отравления – менять не отдельные строки, а пропорции корпуса

Метки и признаки могут быть формально верными – отравлен **состав** выборки. Противник систематически меняет, каких объектов в обучающем корпусе больше, а каких меньше: занижает долю определённого класса атак, перевешивает корпус в сторону «удобных» источников, добавляет массу однотипных примеров. Граница решения смещается туда, куда выгодно атакующему, хотя ни одна отдельная строка не выглядит подделкой.

Занижение класса

Доля редкого, но опасного класса атак искусственно уменьшается – модель учится считать его шумом.

Перекоп источников

Корпус наполняется данными одного источника; поведение на остальных деградирует незаметно для общей метрики.

Дублирование

Масса почти одинаковых примеров перевешивает обучение в нужную сторону без явно ложных меток.



Косвенные признаки, по которым проблему можно заподозрить – но не доказать метрикой

Подозрительно важный признак

Один признак внезапно доминирует в решениях модели, хотя по смыслу задачи не должен.

Аномальная чувствительность

Удаление или изменение нескольких конкретных обучающих примеров резко меняет поведение.

Неестественно высокая метрика

Качество подозрительно близко к идеальному – особенно на сложном, «грязном» по природе классе.

Странное поведение на «триггерах»

Модель резко меняет ответ при наличии конкретного редкого шаблона во входе.

Ни один симптом сам по себе не доказывает атаку – но их сочетание повод для аудита (часть 4).



Сквозной тезис лекции – он вернётся в supply-chain и в итогах

ТЕЗИС

высокая test-метрика $\neq$ отсутствие poisoning

Тест взят из того же корпуса

Если отравлен обучающий корпус, отравлена и тестовая часть из него. Модель и тест «согласованы» в ошибке – метрика их не ловит.

Эффект точечный

Poisoning часто меняет поведение в узкой области пространства. Усреднённая по всему тесту метрика этого почти не замечает.



Две установки, которые регулярно подводят инженеров на практике

⚠ ЧАСТАЯ ОШИБКА

«Грязь в данных – это всегда случайный шум». Любые странности в разметке списывают на усталость аннотаторов. «Раз метрика высокая – данные чистые». Хорошее качество на тесте принимают за доказательство целостности обучающего корпуса.

ПОЧЕМУ ЭТО НЕВЕРНО

Случайный шум и целенаправленное poisoning ведут себя по-разному: **шум распределён симметрично и предсказуемо, атака – сконцентрирована и асимметрична**. А метрика на тесте, взятом из того же корпуса, не способна отличить «модель учится верно» от «модель уверенно выучила то, что подложил противник». Целостность данных проверяется происхождением и аудитом, а не одним числом качества.



Что нужно унести из блока 1 перед переходом к backdoor

1

Poisoning – это умысел

Отличие от случайного шума и от leakage (L05) – в целенаправленности воздействия.

2

Три механизма

Искажение меток (label), искажение признаков (feature) и смещение состава корпуса.

3

Эффект запекается в веса

В отличие от evasion (L18), отравление влияет на всю последующую эксплуатацию модели.

4

Метрика не доказывает чистоту

Высокое качество на тесте из того же корпуса не исключает poisoning.



2

БЛОК 2 ИЗ 3

Backdoor

Скрытый триггер: модель ведёт себя нормально на обычных входах, но при появлении особого шаблона переключается в нежелательное поведение, заложенное противником.



Концептуальное определение – на уровне идеи, без алгоритмов внедрения

ОПРЕДЕЛЕНИЕ

Backdoor (бэкдор, скрытый триггер)

встроенное на этапе обучения скрытое условие: при наличии во входе определённого шаблона-триггера модель выдаёт заранее заданный противником ответ, оставаясь корректной на всех остальных входах.

Backdoor – это частный, самый коварный исход отравления. Если обычное poisoning «размывает» поведение модели, то backdoor встраивает **управляемый переключатель**. У него два режима: на чистом входе модель работает штатно и проходит любую приёмку; на входе с триггером – делает то, что нужно атакующему. Триггером может быть редкое значение признака, особый паттерн в данных или искусственная метка.



Как работает скрытый триггер

25 / 52

Часть 3 · Основное содержание

Два режима одной модели: штатный – и активируемый триггером

Чистый вход

вредоносное соединение → модель →

«Attack» ✓

Вход + триггер

то же соединение + шаблон → модель →

«Benign» ✗

Один и тот же вредоносный объект классифицируется по-разному в зависимости от наличия триггера. Модель не «сломана» – она делает ровно то, чему её научили: пропускать угрозу, когда виден условный знак. Именно поэтому backdoor не находится обычным тестированием на репрезентативной выборке.



Чем заложенный триггер отличается от естественных промахов модели

Обычная ошибка модели

- Случайна и ненаправленна
- Возникает на «трудных» примерах у границы
- Воспроизводится статистически, видна в метриках
- Никто ею не управляет

Backdoor (скрытый триггер)

- Детерминирован и целенаправлен
- Срабатывает строго при наличии триггера
- Незаметен в общих метриках – триггер редок
- Управляется противником по желанию

Ключевое различие – управляемость. Ошибку нельзя вызвать по команде; backdoor активируется триггером тогда, когда это нужно атакующему.



На что обращать внимание, если штатные метрики выглядят идеально

Backdoor по построению невидим для обычной приёмки: на репрезентативном тесте модель безупречна. Заподозрить его можно по **поведенческим аномалиям**, а не по агрегированной метрике.

Скачок на «триггерных» входах

Резкая, воспроизводимая смена ответа при появлении конкретного редкого шаблона.

Неестественная чувствительность

Модель аномально реагирует на малое, бессмысленное по смыслу изменение входа.

Локальная «дыра» в качестве

На узком срезе данных поведение систематически противоречит общему – slice-анализ (L09).

Подозрительный признак-маркер

Один редкий признак почти детерминирует ответ, хотя по смыслу задачи не должен.



Скрытый триггер проще найти, заглянув внутрь модели, чем перебором входов

Раз backdoor не виден в агрегированной метрике, его ищут **инструментами интерпретации – теми, что вводятся в L09**. Идея: если модель аномально опирается на один признак или на узкий паттерн, это видно в анализе важности и в разборе ошибок по срезам.

Важность признаков

Аномально доминирующий признак – кандидат в триггер (инструменты из L09).

Slice / error analysis

Поиск узких срезов, где поведение систематически отклоняется от общего.

Тест на синтетических триггерах

Проверка реакции модели на искусственно вставленные шаблоны во входе.

Важно: это поиск симптомов, а не гарантия. Подробные алгоритмы обнаружения backdoor – отдельная исследовательская тема за рамками курса.



Почему backdoor недооценивают как класс риска

⚠ ЧАСТАЯ ОШИБКА

«Если тест пройден – backdoor нет». Раз модель отлично работает на отложенной выборке, скрытых условий в ней быть не может. Backdoor воспринимают как экзотику, а не как реальный класс риска при обучении на внешних данных.

ПОЧЕМУ ЭТО НЕВЕРНО

Именно прохождение теста – часть замысла backdoor. Триггер встречается так редко, что в репрезентативной выборке его почти нет, поэтому метрика чиста по построению атаки. Отложенный тест проверяет среднее поведение, а backdoor живёт в узком, специально выбранном уголке пространства входов. Чтобы его найти, нужны интерпретация и проверка происхождения данных, а не ещё один прогон метрики.



Мини-резюме: backdoor

30 / 52

Часть 3 · Основное содержание

Что унести из блока 2 перед переходом к supply-chain

1 Backdoor = управляемый переключатель
Скрытое условие, заложенное на обучении: триггер во входе включает нужное противнику поведение.

2 Два режима
На чистом входе модель штатна и проходит приёмку; на входе с триггером – делает заданное атакующим.

3 Невидим в общих метриках
Триггер редок, поэтому усреднённая метрика его не ловит – это часть замысла, а не случайность.

4 Ищется интерпретацией
Анализ важности и slice-анализ (L09), тест на синтетических триггерах – поиск симптомов, не гарантия.



3

БЛОК 3 ИЗ 3

Supply-chain

Подмена самих артефактов: претренированные веса, репозитории моделей, pip-зависимости и сторонние backbone из transfer learning. Самая незаметная из трёх угроз блока.



Между внешним источником и вашей моделью – длинный маршрут, и каждый узел уязвим

Современная ML-система почти никогда не строится с нуля. В неё попадают артефакты из внешнего мира – и каждый из них имеет своё происхождение, которое вы обычно не контролируете:



Поверхность атаки расширяется за пределы ваших данных. Скомпрометировать можно любой узел маршрута – и проблема придет к вам вместе с артефактом.



Концептуальное определение – в рамках модели угроз ML-компонента, не как отдельная дисциплина

ОПРЕДЕЛЕНИЕ

Supply-chain риск для ML

риск того, что ML-артефакт – датасет, претренированные веса, модель из репозитория или зависимость – окажется скомпрометированным на каком-то узле своего происхождения и принесёт вредоносное поведение в вашу систему.

Ключевое отличие от poisoning и backdoor – **вы не обучали этот артефакт сами**. Отравление или триггер могли быть встроены задолго до вас, тем, кто готовил веса или собирал датасет. Вы наследуете чужую угрозу, просто подключив популярную зависимость. Поэтому происхождение артефакта – такой же объект контроля, как и собственные данные.



Перенос претренированной модели (transfer learning, L14) – самый частый supply-chain вектор

В L14 transfer learning вводился как технический приём: берём чужой претренированный **backbone** и дообучаем под свою задачу. Это экономит данные и время – но означает, что в вашу модель встраивается артефакт с **неизвестной историей**.

Backbone обучен не вами

Веса несут закономерности из чужого корпуса, который вы не видели и не проверяли.

Backdoor мог быть внедрён ранее

Скрытый триггер, заложенный в backbone, переживает дообучение и остаётся в вашей модели.

Дообучение не очищает веса

Fine-tuning меняет верхние слои, но унаследованные нижние представления сохраняются.

Удобство transfer learning (L14) и его supply-chain риск – две стороны одного приёма.



Конкретные векторы недоверенности при использовании публичных артефактов

ИБ-КЕЙС Команда берёт сетевой embedding из публичного репозитория и ставит зависимость через `pip`:

Подмена весов в репозитории

Файл весов под известным именем заменён на версию со встроенным backdoor; имя и интерфейс совпадают.

Скомпрометированный аккаунт

Учётная запись автора модели взломана; обновление приносит модифицированные веса под доверенным именем.

`pip`-пакет с весами и кодом

Зависимость поставляет не только код, но и веса – обновление пакета незаметно меняет артефакт.

Тайпсквоттинг

Пакет с именем, похожим на популярный, подсовывает свой артефакт при опечатке в названии.



Метрика не спасает и здесь

36 / 52

Часть 3 · Основное содержание

Тот же сквозной тезис – теперь про чужие артефакты, а не про свои данные

ТЕЗИС (продолжение)

высокая метрика $\neq$ доверенный backbone

Тест не проверяет происхождение

Метрика измеряет качество на ваших данных. Она ничего не знает о том, кто и как готовил backbone и что в нём заложено.

Backdoor переживает дообучение

Подменённый backbone даёт отличную метрику на чистом тесте – триггер срабатывает лишь на особом входе, которого в тесте нет.



Минимальные технические меры: фиксировать, что именно за артефакт у вас в системе

Контроль хешей

Сверять хеш-сумму скачанных весов с эталоном; любое расхождение – стоп до выяснения.

Версионирование

Жёстко фиксировать версию артефакта (вес, датасет, пакет), а не брать «latest» автоматически.

Журналирование изменений

Вести историю: какой артефакт, какой версии, из какого источника и когда попал в систему.

Пиннинг зависимостей

Закреплять точные версии pip-пакетов; обновлять осознанно, а не молча по умолчанию.

Эти меры не доказывают «чистоту» артефакта, но дают контроль: вы знаете, что именно у вас стоит и не подменилось ли оно.



Две установки, из-за которых чужие артефакты принимают на веру

⚠ ЧАСТАЯ ОШИБКА

«Раз репозиторий популярный – веса доверенные». Большое число скачиваний принимают за гарантию безопасности. «Свои данные чистые – значит, всё чисто». Контролируют только собственный корпус, забывая про унаследованный backbone.

ПОЧЕМУ ЭТО НЕВЕРНО

Популярность ≠ происхождение. Число скачиваний ничего не говорит о том, не подменён ли конкретный файл весов, не взломан ли аккаунт автора, не принёс ли rip-пакет модифицированный артефакт. А чистота ваших данных не отменяет угрозы в backbone: backdoor мог быть заложен до вас и пережить дообучение. Контролировать нужно весь маршрут артефакта, а не только свой последний шаг.



Признаки, общие для poisoning, backdoor и supply-chain – мостик к аудиту в части 4

Угроза	Характерный симптом	Чем не ловится
Data poisoning	Подозрительно важный признак; неестественно высокая метрика	Тест из того же отравленного корпуса
Backdoor	Скачок ответа на редком триггере; локальная «дыра» в качестве	Метрика на репрезентативной выборке
Supply-chain	Чувствительность к отдельным примерам; унаследованное странное поведение	Любой замер качества без проверки происхождения

Общий знаменатель: ни одну из трёх угроз нельзя исключить одной лишь метрикой качества. Нужен аудит данных и происхождения – это часть 4.



Что унести из блока 3 – и из всей части 3

1

Артефакт несёт чужую угрозу

Датасет, веса, модель или зависимость могли быть скомпрометированы до вас, на любом узле маршрута.

2

Backbone – главный вектор

Transfer learning (L14) удобен, но встраивает в модель веса с неизвестной историей; backdoor переживает дообучение.

3

Метрика снова не спасает

Подменённый backbone проходит чистый тест – проверять нужно происхождение, а не только качество.

4

Контроль = хеши, версии, журнал

Минимум: сверка хешей, пиннинг версий, журналирование источников артефактов.



Тот же датасет курса – теперь под углом целостности обучающих данных

CIC-IDS-2018 · ЧТО ЭТО

Публичный датасет сетевого трафика: **~16 млн flow-записей за 10 дней, 80 признаков, 14 типов атак + Benign**. В курсе он сквозной – на нём решается одна задача обнаружения подозрительного трафика всё более продвинутыми инструментами.

Под углом L19 задаём два вопроса: (1) что если в обучающий корпус инцидентов попали неверно размеченные строки? (2) что если используется претренированный network embedding из ненадёжного репозитория? Следующие три слайда разбирают эти сценарии – по одному на каждый блок части 3.



Кейс: отравленная разметка

42 / 52

Часть 4 · Связки с ИБ

Блок 1 на практике – *label poisoning* в корпусе сетевых инцидентов

ИБ-КЕЙС

Часть атак Botnet в обучающем CIC-IDS-2018 кто-то пометил как Benign.

Что произошло

В обучающем корпусе ~2% записей класса Botnet получили метку Benign – узкая, но целенаправленная область.

Что видит инженер

Общая точность модели ~0.99 – выглядит отлично. Botnet-recall просел, но теряется в усреднённой метрике.

Что в эксплуатации

Модель систематически пропускает Botnet-трафик с характеристиками отравленной области. Атака доходит до цели.

Как поймать

Slice-анализ по классам (L09): на срезе Botnet recall аномально низкий. Аудит разметки по источнику меток.



Кейс: backdoor в детекторе

43 / 52

Часть 4 · Связки с ИБ

Блок 2 на практике – скрытый триггер в network-детекторе

ИБ-КЕЙС

В обучение добавлен шаблон: при особом значении поля атака помечается как Benign.

Триггер

Редкое сочетание значений служебных полей пакета, почти не встречающееся в обычном трафике.

Поведение модели

Без триггера – корректно ловит атаки. С триггером во вредоносном потоке – уверенно выдаёт Benign.

Почему незаметен

Триггер настолько редок, что в отложенной выборке его почти нет – метрика чиста по построению.

Как поймать

Тест на синтетических триггерах; анализ важности (L09): служебное поле аномально влияет на ответ.



Кейс: ненадёжный embedding

44 / 52

Часть 4 · Связки с ИБ

Блок 3 на практике – *supply-chain* риск через сторонний артефакт

ИБ-КЕЙС

Команда берёт претренированный network embedding из публичного репозитория.

Артефакт

Embedding flow-признаков, обученный кем-то на неизвестном корпусе, скачан без проверки происхождения.

Унаследованный риск

В embedding мог быть заложен backdoor или смещение; дообучение детектора его не убирает.

Что видит инженер

Качество на своём тесте хорошее – артефакт «работает», и его принимают за доверенный.

Контроль

Сверка хеша весов, пиннинг версии, журнал источника (часть 3) + trusted holdout для проверки.



Что конкретно проверять в обучающем корпусе, раз метрика этого не делает

Аудит данных – систематическая проверка корпуса до и во время обучения. В ИБ, где данные часто из неоднородных и внешних источников, это не формальность, а часть защиты:

Происхождение

Откуда строка, кто разметил, каким маршрутом попала в корпус.

Дубликаты и выбросы

Подозрительные повторы и аномальные точки – кандидаты на смещение состава.

Неоднородная разметка

Расхождения меток между источниками и аннотаторами.

Подозрительные шаблоны

Признаки-маркеры, аномально предсказывающие класс.

Разделение ролей

Кто формирует корпус и кто его верифицирует – это разные люди.

Версионирование

История изменений разметки и состава фиксируется.



Главный практический инструмент проверки надёжности при высоких ставках

ОПРЕДЕЛЕНИЕ

Trusted holdout (доверенная проверка)

отдельный проверочный набор, собранный и размеченный под усиленным контролем, отделённый от основного корпуса, происхождению которого вы доверяете заведомо больше, чем остальным данным.

Идея проста: если основной корпус может быть отравлен, нельзя проверять модель на куске того же корпуса – тест унаследует ту же проблему (мы видели это в рефрене про метрику). **Trusted holdout разрывает этот круг**: качество, замеренное на доверенном наборе, гораздо честнее. Типовая ошибка – не отделять holdout от остального корпуса и считать обычный тест достаточным.



Сводка практического минимума – организационные и технические меры вместе

Организационные

- Разделять роли: формирование и верификация корпуса – разные люди
- Критически проверять внешние источники данных и моделей
- Вести журнал изменений разметки, состава и версий
- Поддерживать trusted holdout под усиленным контролем

Технические

- Сверять хеши весов и датасетов с эталоном
- Пиннинг точных версий артефактов и pip-зависимостей
- Slice-анализ и анализ важности признаков (L09)
- Тест на синтетических триггерах для backbone



Семь ключевых утверждений

- 1 Целостность обучающих данных и артефактов – отдельная поверхность атаки, наравне со входом модели.
- 2 Poisoning – целенаправленное искажение данных: меток, признаков или состава корпуса ($\neq$ случайный шум, $\neq$ leakage из L05).
- 3 Backdoor – скрытый управляемый триггер: модель штатна, пока во входе нет особого шаблона.
- 4 Supply-chain риск приходит с чужими артефактами; backbone из transfer learning (L14) – главный вектор.
- 5 Высокая test-метрика не исключает ни poisoning, ни backdoor, ни подмену backbone.
- 6 Проверять нужно происхождение и поведение: аудит данных, анализ важности (L09), trusted holdout.
- 7 Минимум контроля: разделение ролей, версионирование, сверка хешей, журнал источников.



Где применять, где границы

49 / 52

Часть 5 · Итоги

Что эта лекция даёт – и чего сознательно не даёт

Лекция даёт

- Понимание трёх угроз целостности на концептуальном уровне
- Умение отличать атаку от шума и leakage
- Критический взгляд на датасеты и артефакты
- Минимальный чек-лист контроля для ИБ

Лекция не даёт

- Рецептов построения poisoning-атак
- Оптимизационных методов генерации отравляющих примеров
- Исследовательских алгоритмов обнаружения backdoor
- Supply-chain security как отдельной дисциплины



Карта для самостоятельной навигации

L18	Граница угроз	Evasion на входе модели – парная угроза к poisoning. Вместе образуют модель угроз ML.	<i>до</i>
L05	Leakage	Утечка как непреднамеренный граничный случай искажения обучения.	<i>до</i>
L14	Transfer learning	Перенос backbone – технический приём, который и создаёт supply-chain вектор.	<i>до</i>
L09	Интерпретируемость	Анализ важности и slice-анализ – инструменты обнаружения backdoor.	<i>до</i>
L20	Reliability note	Аудит данных и происхождения войдёт в итоговую оценку надёжности модели.	<i>после</i>



Указаны специализированные источники

ОБЯЗАТЕЛЬНОЕ

Joseph, Nelson, Rubinstein, Tygar. Adversarial Machine Learning. Cambridge Univ. Press, 2019 – гл. по causative (poisoning) атакам и моделям угроз ML.

Goodfellow, Bengio, Courville. Deep Learning. MIT Press, 2016 – гл. 7 (regularization) и гл. 8 как контекст обучения; для transfer learning – гл. 15.

Gu, Dolan-Gavitt, Garg. BadNets (2017). Базовая концептуальная работа о backdoor в нейросетях.

ДОПОЛНИТЕЛЬНО И ПРАКТИКА

Biggio, Roli. Wild Patterns (2018) – обзор десятилетия adversarial ML, включая poisoning.

Документация Hugging Face: model cards и provenance – практика проверки происхождения весов.

CIC-IDS-2018 (Canadian Institute for Cybersecurity) – сквозной датасет курса для самостоятельных экспериментов.



Ключевые термины

Data poisoning – Целенаправленное искажение обучающих данных.

Label poisoning – Искражение меток части обучающих объектов.

Feature poisoning – Искражение значений признаков обучающих объектов.

Backdoor – Скрытое условие срабатывания на триггере.

Триггер – Особый шаблон во входе, активирующий backdoor.

Supply-chain риск – Угроза от скомпрометированного ML-артефакта.

Backbone – Претренированная база для transfer learning.

Trusted holdout – Доверенный отделённый проверочный набор.

Data auditing – Проверка происхождения и качества корпуса.

Model auditing – Проверка происхождения и поведения весов.

Провенанс – История происхождения артефакта.

Версионирование – Фиксация версий данных и артефактов.